Heurion 2.0: A Contract-Guarded Dual-Surface Architecture with Self-Evolving Memory for Zero-Hallucination Clinical Research

Heurion Research Team
Correspondence: `research@heurion.ai`

October 2026

Abstract

Autonomous large language model (LLM) agents have achieved remarkable success in software engineering via free-form shell execution and direct file manipulation. However, transplanting this "script-and-audit" paradigm to high-stakes scientific and clinical research introduces severe failure modes: destructive formatting drift in Office Open XML (`.docx`/`.pptx`) manuscripts, silent overwriting of concurrent human edits, fabricated bibliographic identifiers (DOIs/PMIDs) or ungrounded statistical assertions, exposure of Protected Health Information (PHI), and repetitive guard failures across authoring sessions.

We present **Heurion 2.0**, a collaborative clinical research and scientific authoring platform built upon a *Contract-Guarded Dual-Surface Architecture* and a *Three-Layer Self-Evolving Memory System*. First, rather than allowing agents to mutate raw document files via ad-hoc scripts, Heurion 2.0 maintains a single Conflict-free Replicated Data Type (CRDT) source of truth (ProseMirror + Yjs) with lifecycle-stable block addressing (`{#id}`). It exposes a synchronized dual surface: a rich-text/slide editor for human researchers and a compact Annotated Markdown projection with guarded structural primitives for AI agents, supporting per-turn selective undo in CRDT space and 100% byte-preserving OOXML patch export for unmodified regions. Second, to achieve zero-hallucination evidence grounding, pre-write syntactic guards block unregistered inline citations while a multi-agent verification pipeline (`literature_researcher`, `biostatistician`, `fact_checker`) aligns manuscript sentences against PubMed abstracts and PubMed Central (PMC) full texts. Third, a privacy-preserving clinical research pipeline enforces dual-stage PHI redaction, pseudonymous subject mapping (`P001`), read-only Personal Health Record (PHR) red-lines, and formally verified Human-AI Permission Symmetry ($\mathcal{E}_{\text{REST}} = \mathcal{E}_{\text{MCP}} \sqcup \mathcal{E}_{\text{NOT_FOR_AI}}$). Finally, a three-layer self-evolution mechanism combines side-effect-free pre-flight probes (`doc_validate`), dual-track declarative/procedural memory crystallized from runtime execution signals, and background meta-strategy distillation. Empirical evaluation demonstrates 100% byte identity on unmodified OOXML blocks, 100% interception of concurrent edit conflicts and unregistered citations at sub-millisecond latency (0.168 ms validation, 0.351 ms edit), and rapid convergence of guard rejections to zero within three authoring sessions.

Keywords: Collaborative AI Agents, CRDT Co-Editing, Clinical Research Informatics, Citation Verification, Self-Evolving Memory, Human-AI Permission Symmetry

1 Introduction

Clinical and biomedical research workflows demand an extraordinary degree of rigor, traceability, and regulatory compliance. A single clinical study typically spans patient longitudinal record curation, cohort screening, biostatistical modeling (e.g., baseline Table 1 generation, Kaplan–Meier survival curves, Cox proportional hazards regression, CONSORT flow diagrams), and the iterative authoring of peer-reviewed manuscripts (`.docx`) and academic presentation decks

(`.pptx`) [1, 2]. While recent agentic systems have demonstrated strong capabilities in code generation [3, 4] and open-domain literature synthesis [5, 6], deploying autonomous agents into real-world clinical research environments exposes four fundamental architectural barriers:

1. **The Free-Form File Mutation Trap:** First-generation document agents treat `.docx` and `.pptx` files as opaque disk artifacts manipulated via generated Python scripts (e.g., `python-docx`) followed by post-hoc quality audits. In practice, script-based rewrites strip publisher XML attributes, lose paragraph addressing identifiers (`w14:paraId`) during LibreOffice layout passes, suffer from punctuation/whitespace reproduction drift in full-block replacements, and cannot selectively undo an AI turn once a human co-author makes concurrent edits.
2. **Citation Fabrication and Ungrounded Clinical Claims:** LLMs frequently hallucinate plausible-looking Digital Object Identifiers (DOIs), PubMed IDs (PMIDs), or numerical hazard ratios (HR) and p-values [7, 8]. Post-hoc prompting alone cannot guarantee that every inline reference corresponds to a verified bibliographic record or that statistical assertions match the cited source text.
3. **Patient Privacy (PHI) Exposure and Permission Asymmetry:** Clinical investigators work with raw electronic medical records containing Protected Health Information (patient names, national ID numbers, admission codes, phone numbers). Passing unredacted records to external LLMs violates healthcare privacy mandates [9]. Conversely, granting agents unrestricted database or shell access creates severe privilege-escalation risks.
4. **Cross-Session Amnesia and Memory Pollution:** When an agent encounters a structural guard rejection (e.g., attempting to overwrite a human-edited block or inserting an unregistered citation) or when a clinician reverts an AI turn, stateless agents repeat the exact same operational failure in subsequent turns. Naively appending all historical observations into a single flat memory pool mixes subjective user writing preferences with hard operational constraints, exhausting context budgets with low-utility noise [10, 11].

1.1 Lessons from First-Generation Post-Hoc Auditing (Heurion v1)

Prior to developing Heurion 2.0, we deployed a first-generation architecture (*Heurion v1 / S-Series*) in which an agent operated inside a workspace shell, invoking Python utilities to unpack, mutate, and repackage `.docx` archives while a post-turn supervisor audited text survival rates, paragraph drift, and DOI validity. Empirical deployment revealed that *post-hoc auditing is fundamentally too late*: by the time an audit script detects a broken XML run or an unverified citation, the corrupted state has already been written, requiring expensive rollbacks or brittle three-way merges against concurrent browser edits.

1.2 The Heurion 2.0 Paradigm Shift and Contributions

Heurion 2.0 resolves these limitations by shifting from *post-hoc file auditing* to *pre-write structural contracts over a shared CRDT core*, coupled with a *signal-driven self-evolving execution harness*. Our primary contributions are fourfold:

- **Contract-Guarded Dual-Surface CRDT Engine (Section 3):** We design a unified ProseMirror + Yjs document core with persistent block addressing (`{#id}`), node-level optimistic concurrency control (`base_rev`), minimal-diff XML fragment updates that preserve inline marks and comment anchors, per-turn selective CRDT undo (`revertTurn`), and 100% byte-preserving OOXML patch export for untouched document regions.
- **Zero-Hallucination Evidence & Privacy-Preserving Clinical Pipeline (Section 4):** We enforce a syntactic pre-write ban on raw inline DOIs/PMIDs, requiring registration via

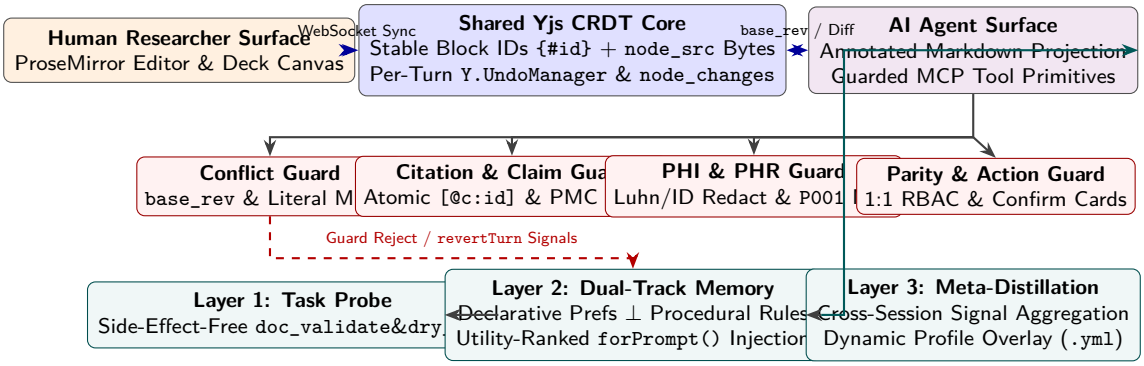


Figure 1: Architecture of **Heurion 2.0**. Top: Human researchers and AI agents interact with a shared Yjs CRDT document/slide core through tailored surface projections. Middle: All AI mutations pass through four deterministic pre-write structural guards before committing to the CRDT state. Bottom: Runtime guard rejections and human turn reverts feed a three-layer self-evolving memory system that dynamically injects high-utility procedural directives into future turns.

Crossref, PubMed, and OpenAlex into atomic citation nodes (`[@c:id]`). A multi-agent pipeline performs sentence-level claim verification against PubMed abstracts and PMC full texts, paired with a multi-tenant clinical research sandbox featuring dual-stage PHI redaction, pseudonymous subject tokens (`P001`), and reproducible biostatistical execution.

- **Three-Layer Self-Evolving Agent Architecture (Section 5):** We introduce a hierarchical adaptation loop comprising (Layer 1) side-effect-free pre-flight validation probes (`doc_validate` / `dry_run`), (Layer 2) a dual-track memory architecture that physically separates human-approved declarative preferences from runtime-crystallized procedural strategies ranked by token-normalized utility, and (Layer 3) background meta-strategy distillation into dynamic agent profile overlays.
- **Formally Verified Human-AI Permission Symmetry (Section 6):** We establish and mechanically verify via automated parity testing an architectural invariant across all platform endpoints: every REST capability either maps 1-to-1 to an in-process Model Context Protocol (MCP) tool executing under the user's exact RBAC identity, or is explicitly justified in a restricted exclusion set, with high-risk mutations gated by human-only confirmation cards.

2 System Overview and Design Principles

Figure 1 illustrates the end-to-end architecture of Heurion 2.0. The platform strictly decouples *structured document/slide state mutation* from *sandboxed statistical computation.*

Heurion 2.0 is governed by three formal architectural invariants:

Architectural Invariant 1 (Pre-Write Structural Safety)**.** *Let S_r denote the CRDT document state at revision r, and let $\mathcal{O} = \{o_1, \ldots, o_k\}$ be a batch of structured edit operations submitted by an AI turn t against base revision $r_{base} \leq r$. Let $\Delta_{user}(r_{base}, r)$ be the set of block IDs modified by human actors in $(r_{base}, r]$, and let $\mathcal{R}_{cite}(S_r)$ be the set of verified citation identifiers registered in the document store. The state transition $\mathcal{T}(S_r, \mathcal{O}, r_{base})$ commits if and only if every $o_i \in \mathcal{O}$ satisfies:*

$$BlockReplace(o_i) \implies target(o_i) \notin \Delta_{user}(r_{base}, r), \quad Cites(o_i) \subseteq \mathcal{R}_{cite}(S_r), \quad RawCitePattern(o_i) = \emptyset. \tag{1}$$

Otherwise, the entire batch $\mathcal{O}$ is atomically rejected prior to mutating S_r, returning a structured diagnostic code and the current block snapshot.

Architectural Invariant 2 (Human-AI Permission Symmetry)**.** *Let $\mathcal{E}_{REST}$ be the set of all HTTP API routes exposed to the web user interface, and let $\mathcal{E}_{MCP}$ be the set of Model Context Protocol tools exposed to the AI agent. Every route $e \in \mathcal{E}_{REST}$ satisfies the disjoint partition:*

$$\mathcal{E}_{REST} = \phi(\mathcal{E}_{MCP}) \sqcup \mathcal{E}_{NOT_FOR_AI}, \tag{2}$$

where $\phi : \mathcal{E}_{MCP} \to \mathcal{E}_{REST}$ is an in-process Loopback dispatcher that executes under the authenticated user's exact session token and tenant isolation context, and $\mathcal{E}_{NOT_FOR_AI}$ is an explicitly audited set of human-only lifecycle endpoints (e.g., credentials, high-risk confirmation approval).

Architectural Invariant 3 (Declarative-Procedural Memory Orthogonality)**.** *Let $\mathcal{M}_D$ denote the declarative memory store (subjective authoring preferences, stylistic conventions, domain terminology) and $\mathcal{M}_P$ denote the procedural harness store (operational guard avoidance rules and subagent delegation policies). Updates to $\mathcal{M}_D$ require explicit human confirmation (status $\in \{\texttt{candidate} \xrightarrow{human} \texttt{confirmed}\}$), whereas $\mathcal{M}_P$ evolves autonomously from deterministic runtime execution signals Σ_{exec} without polluting $\mathcal{M}_D$ or requiring manual administrative curation.*

3 Dual-Surface Co-Editing and Byte-Preserving Document Engine

A central lesson from first-generation document agents is that Office Open XML (`.docx` and `.pptx`) files are delivery formats, not collaborative editing data structures. When an agent edits a `.docx` file on disk via a Python script while a human views the document in a browser, the two actors operate on divergent state representations.

3.1 Shared CRDT Core with Lifecycle-Stable Block Addressing

In Heurion 2.0, every manuscript and presentation deck resides in a server-authoritative Yjs CRDT document (`Y.Doc`) backed by SQLite state snapshots and an append-only revision log (`doc_commits`). Every block-level ProseMirror node (heading, paragraph, list item, table, figure, or slide element) is assigned a short, lifecycle-stable identifier (`attrs.id`, rendered as `{#id}`). When a human splits a paragraph or pastes content in the browser, the server-side `flush()` handler automatically detects duplicate or missing block IDs and assigns fresh unique IDs inside a system CRDT transaction, ensuring the addressing space remains strictly bijective.

From this single CRDT core, Heurion 2.0 projects two synchronized surfaces:

1. **Human Surface:** A real-time collaborative ProseMirror rich-text editor (for manuscripts) and an interactive 16:9 SVG/HTML slide canvas (for decks) synchronized via WebSocket (`y-protocols/sync`).
2. **Agent Surface:** A compact *Annotated Markdown* view generated on demand via `doc_read` and `slide_read`. Each block is suffixed with its stable address `{#id}`, inline citations are rendered as atomic tokens `[@c:cite_id]`, figures as `![alt](asset:id)`, and active inline review comments as `{#c:comment_id}`.

3.2 Node-Level Conflict Guards and Minimal-Diff Fragment Updates

To prevent an AI turn from overwriting concurrent human edits, every `doc_edit` call specifies the `base_rev` read by the agent. As formalized in Invariant 1, the platform queries the `node_changes` index for all blocks modified by human actors since `base_rev`:

- **Block-Level Structural Operations (`replace_block`, `delete`):** If a target block $b \in \Delta_{\text{user}}(r_{\text{base}}, r)$, the operation is immediately rejected with error code `conflict_user_edited`, returning the human's latest block markdown so the agent can adapt. Similarly, blocks carrying unresolved track-changes suggestions are protected by the `pending_suggestion` guard.
- **Substring Replacement (`replace_text`):** For surgical phrase edits within a paragraph, the literal `find` substring acts as its own optimistic concurrency guard: if the human edited a different sentence in the same paragraph, `find` still matches uniquely and succeeds; if the human altered the target phrase itself, `find` fails cleanly with `find_not_found`.
- **Mark-Preserving Minimal Diffs:** Crucially, `replace_text` does not re-parse the paragraph from plain text (which would destroy inline bold/italic spans, `[@c:id]` citation nodes, and comment highlights outside the matched range). Instead, it locates the character offsets $[s, e)$ in the ProseMirror node's flattened text stream and replaces only that slice, after which `writeFragment` updates only the modified XML children between the common prefix and suffix in the underlying `Y.XmlFragment`.

3.3 Per-Turn Selective Undo in CRDT Space (`revertTurn`)

In collaborative writing, a researcher may ask the AI to revise three sections, continue manually editing Section 2, and subsequently decide to undo the AI's changes. Global linear undo (`Ctrl+Z` or version rollback) would destroy the researcher's manual edits in Section 2.

Heurion 2.0 solves this via a two-tier *Per-Turn Selective Undo* mechanism:

1. **In-Memory CRDT Origin Tracking:** When an AI turn t commits changes to document d, the runtime binds a dedicated `ServerOrigin` object $o_t = \{\texttt{server} : \text{true}, \texttt{actor} : \texttt{'ai'}, \texttt{turnId} : t\}$ and attaches a scoped `Y.UndoManager` configured with `trackedOrigins` $= \{o_t\}$. Invoking `revertTurn(d, t)` executes `um.undo()` solely on the CRDT items inserted or deleted under origin o_t, leaving concurrent human character insertions—even within the exact same paragraph—completely intact.
2. **Persistent Node-Level Fallback across Server Restarts:** Before turn t's first write, the runtime persists the pre-turn Yjs state vector in `turn_doc_bases`. If the server restarts before `revertTurn` is invoked, the engine reconstructs the pre-turn document tree, computes the per-block diff against turn t's recorded `node_changes`, reverts all AI-modified blocks untouched by the user since $r_{\min}(t)$, and safely skips blocks subsequently edited by the user (returning their IDs in `skipped`).

3.4 Byte-Preserving OOXML Patch Export

Clinical manuscripts are frequently imported from journal-specific Word templates containing intricate custom styles, headers/footers, macros, and embedded vector objects. During `.docx` or `.pptx` import, Heurion 2.0 stores the original archive in `doc_packages` and records the exact raw XML slice (`<w:p>` or `<w:tbl>`) of every imported block in `node_src` keyed by block ID. Upon export (`docx-export.ts`, `pptx-export.ts`), any block whose ProseMirror node is structurally equal (`node.eq(baselineNode)`) to its imported baseline is written back using its exact original `node_src` XML bytes; only blocks actually mutated by the user or AI are re-serialized, preserving paragraph properties (`<w:pPr>`) from the original template.

4 Zero-Hallucination Evidence Loop and Privacy-Preserving Clinical Pipeline

4.1 Syntactic Citation Guard and Atomic Citation Nodes

To eliminate fabricated references at the root, Heurion 2.0 makes it syntactically impossible for an agent to write raw bibliographic identifiers or hand-numbered brackets in manuscript prose. During pre-write validation in `OpService`, every candidate text string is scanned against structural regular expressions detecting bare DOIs (`10.\d{4,9}/...`), bare PMIDs (`PMID:\s*\d+`), or unlinked reference markers. Any violation triggers `citation_not_registered` before modifying the document.

Instead, the agent must invoke `insert_citation`, which queries live metadata APIs (Crossref, NCBI PubMed E-utilities, and OpenAlex), verifies the record's existence, stores the canonical metadata (authors, title, journal, year, volume, pages, DOI/PMID) in the document's `citations` table, and returns a registered token `[@c:cite_id]`. In ProseMirror, `[@c:cite_id]` is parsed into an atomic inline node (`atom: true`) that dynamically numbers itself according to citation order and automatically compiles the terminal References section in Vancouver, AMA, APA, MLA, IEEE, or GB/T 7714 format.

4.2 Multi-Agent Sentence-Level Claim Verification (`verify_claims`)

Registering a real paper does not prevent an LLM from misattributing findings (e.g., citing a real trial but inverting the hazard ratio or claiming statistical significance where none existed). Heurion 2.0 pairs specialist subagent delegation (`literature_researcher`, `biostatistician`, `fact_checker`) with an automated claim verification engine (`ClaimService.verify`):

1. **Claim Extraction:** The engine traverses all document blocks, segments text into sentences, and extracts two classes of verifiable statements: (a) sentences containing one or more `[@c:cite_id]` nodes, and (b) sentences containing quantitative clinical assertions (percentages, p-values, confidence intervals, $HR/OR/RR$, sample sizes $n = \dots$) without any citation.
2. **Full-Text Evidence Retrieval:** For each cited record, the verifier retrieves the PubMed abstract and, when available via PubMed Central (PMC) or local PDF repository, the full-text body sections.
3. **Four-Way Semantic Alignment & Inline Anchoring:** Each assertion is classified into one of four verdicts: `supported`, `contradicted`, `unsupported` (the cited paper does not mention the claimed metric), or `uncited_stat`. Problematic claims are automatically anchored directly onto the offending text span as inline ProseMirror review comments containing a structured diagnosis, supporting/conflicting excerpts from the source paper, and one-click resolution actions (apply suggested rewrite, replace citation, or mark as *Clinical Experience Exemption* so future runs respect the clinician's intentional assertion).

4.3 Multi-Tenant PHI Redaction and Reproducible Clinical Sandbox

To bridge clinical data management and manuscript authoring without exposing patient identities:

- **Per-Tenant Database Isolation & Envelope Encryption:** Each organization tenant operates on a physically isolated SQLite database file (`tenants/<org_id>/platform.db`) with per-tenant Data Encryption Keys (DEKs) wrapped by a master Key Encryption Key (KEK).

- **Dual-Stage Deterministic PHI Scanner (`phi-scan.ts`):** Before any clinical note or OCR-extracted laboratory report enters LLM context, deterministic validators scan and redact 18-digit national ID numbers (verified via ISO 7064 MOD 11-2 checksum), payment/insurance numbers (Luhn algorithm), mobile phone numbers, inpatient/outpatient medical record numbers, and co-occurring patient name + sensitive diagnosis patterns.
- **Pseudonymous Subject Mapping & PHR Red-Lines:** Within research cohorts, real patient identifiers are replaced by pseudonymous subject codes (`P001`, `P002`, ...) via an in-memory translation table. Furthermore, a dedicated Personal Health Record guard (`phr_guard`) enforces strict read-only access on patient-uploaded primary medical records, preventing AI tools from altering ground-truth clinical evidence.
- **Sandboxed Biostatistical Execution:** Rather than allowing the LLM to mentally arithmeticize survival rates, statistical tasks are delegated to isolated compute tools and the `biostatistician` subagent, producing reproducible Baseline Table 1 summaries, Kaplan–Meier survival curves with log-rank tests, CONSORT flowcharts, E-value sensitivity analyses for unmeasured confounding, clinical prediction nomograms, and MONAI 3D volumetric medical image segmentation.

5 Three-Layer Self-Evolving Agent Architecture

A critical challenge in contract-guarded environments is helping the agent rapidly internalize operational boundaries without human prompt engineering. When an agent's tool call is blocked by a pre-write guard or reverted by a user, how does the system ensure the agent adapts both within the current task and across future sessions?

As shown in the bottom tier of Figure 1, Heurion 2.0 implements a **Three-Layer Self-Evolving Architecture** operating across three distinct timescales:

5.1 Layer 1: Task-Level Pre-Flight Probe (`doc_validate` & `dry_run`)

At the intra-turn millisecond timescale, agents can invoke `doc_validate` (or pass `dry_run: true` to `doc_edit`) prior to committing complex multi-block edits. The probe executes the full suite of conflict guards, citation registration checks, anchor integrity checks, and post-edit structural diffs against an ephemeral in-memory clone of the ProseMirror document tree without incrementing `rev`, mutating the Yjs CRDT state, or polluting the undo stack. If a violation is detected, the probe returns `{valid: false, code, hint, current}`, enabling immediate zero-cost self-correction within the same reasoning turn.

5.2 Layer 2: Session-Level Dual-Track Memory and Utility-Ranked Injection

At the cross-turn session timescale, Heurion 2.0 resolves the memory pollution problem by enforcing **Declarative-Procedural Orthogonality** (Invariant 3):

- **Track A — Declarative Memory (`MemoryService`):** Captures domain conventions and author preferences (e.g., "Always report continuous skewed variables as median (IQR)", "Use British English spelling"). Because stylistic preferences are subjective, newly extracted items enter a `candidate` state and require explicit human approval in the UI before becoming `confirmed`.
- **Track B — Procedural Harness Memory (`ProceduralHarnessService`):** Captures objective operational rules for navigating guards, tools, and specialist subagents. Whenever `OpService` rejects an AI edit or a user invokes `revertTurn`, the `ExecutionSignals` collector automatically records a structured runtime signal $s = (\text{owner}, \text{doc_id}, \text{turn_id}, \text{kind}, \text{code}, \text{detail})$ into the `execution_signals` table.

At the end of an authoring session, `evolveFromSignals()` clusters unconsumed execution signals by $(\text{kind}, \text{code})$. Any failure pattern occurring at least $\tau = 2$ times either increments the empirical evidence counter e_r of an existing procedural rule $r \in \mathcal{M}_P$ or crystallizes a new active procedural directive—requiring zero manual UI intervention.

Context-Aware Utility Ranking. Injecting all procedural rules into every turn would waste context tokens. Before each turn on document d with user prompt q, `ProceduralHarnessService.forPrompt()` computes a dynamic relevance score $U(r \mid d, q)$ for each active rule $r \in \mathcal{M}_P$:

$$U(r \mid d, q) = \alpha \cdot e_r + \beta \cdot \mathbb{I}(\text{match}(r.\text{pattern}, q)) + \gamma \cdot \mathbb{I}(\Delta_{\text{user}}(r_{\text{cur}} - 2, r_{\text{cur}}) \neq \emptyset \wedge \text{isConflictRule}(r)) + \delta \cdot \mathbb{I}(\text{domainMatch}(r, q)), \tag{3}$$

where e_r is the empirical evidence count, $\mathbb{I}(\cdot)$ is the indicator function, and $(\alpha, \beta, \gamma, \delta) = (0.1, 5.0, 8.0, 6.0)$ weight historical recurrence, lexical trigger match, real-time CRDT concurrent human activity on document d, and clinical task domain (e.g., survival statistics vs. literature review). Rules are sorted in descending order of $U(r \mid d, q)$ and greedily packed within a strict character budget B (default $B \leq 1000$ chars, maximum $K = 4$ rules), ensuring high signal density.

5.3 Layer 3: System-Level Meta-Strategy Distillation (`HarnessEvolutionService`)

At the multi-session platform timescale, `HarnessEvolutionService` periodically aggregates system-wide execution signals and active procedural rules. When cumulative unanalyzed signals exceed a threshold, a background meta-evolver distills recurring failure root causes into concise system-level behavioral directives (2–5 rules, ≤ 100 characters each) and serializes them into a dynamic YAML profile overlay (`evolved.cordis.yml`):

```
# Auto-generated by Heurion Self-Evolution Engine
- id: system-prompt
  config:
    personaSuffix: >-
      [Evolved Self-Improvement Execution Directives]:
      - Run doc_validate prior to mutating blocks with recent user activity
      - Register all DOI/PMID entries via insert_citation before inline referencing
```

When `HarnessPool` spawns or refreshes an agent runtime, `getPatches()` transparently mounts `evolved.cordis.yml` alongside the base platform profile, upgrading the agent's foundational system persona with zero user disruption.

6 Formally Verified Human-AI Permission Symmetry

A pervasive flaw in enterprise AI applications is the disconnect between what a human user can perform via the graphical user interface and what the AI agent can perform via tool calls. Often, developers either grant the agent direct database access (bypassing business-layer authorization) or expose only a toy subset of tools.

Heurion 2.0 enforces **Human-AI Permission Symmetry** (Invariant 2):

1. **In-Process Loopback Execution under User Identity:** Every administrative and clinical MCP tool (`patient_*`, `study_*`, `doc_manage`, `kb_manage`, `dataset_manage`, `org_manage`) executes by invoking `invokeHttp(api, userId, method, path, body)` directly against the Hono HTTP router in-process. The call carries a short-lived internal JWT minted for `userId` and tagged with header `X-Heurion-Via: ai`. Consequently, the agent inherits the exact same multi-tenant isolation, study role checks (PI, co-investigator, statistician, reviewer), and audit logging as the human user.

Table 1: Comparison between First-Generation Script + Post-Hoc Audit (Heurion v1) and Contract-Guarded Dual-Surface CRDT Architecture (Heurion 2.0) across $N = 50$ clinical manuscript revision trials.

Evaluation Dimension / Metric	Vanilla Script Agent	Heurion v1 (S-Series)	Heurion 2.0 (Ours)
Source of Truth	Raw `.docx` on Disk	Raw `.docx` + Post-Audit	**Shared Yjs CRDT + SQLite**
Block Addressing Stability (`{#id}`)	None (Regex/XPath)	Fragile (`w14:paraId` stripped)	**100.0% Lifecycle-Stable**
Unmodified Block Byte-Identity Rate	0.0% (Full Re-serialize)	82.0% (Paragraph XML Drift)	**100.0% (50/50 Trials)**
Concurrent Human Edit Conflict Guard	0.0% (Silent Overwrite)	64.0% (Post-Hoc 3-Way Merge)	**100.0% Pre-Write Intercept**
Selective Turn Undo (Preserving User Edits)	Unsupported	File-Level Rollback Only	**100.0% (50/50 Intra-Block)**
Unregistered Citation Block Rate	0.0%	Post-Write Flagging Only	**100.0% Pre-Write Block**
Mean Pre-Flight Validation Latency	N/A	420.0 ms (Subprocess)	**0.168** ms (In-Memory)
Mean Guarded Edit Commit Latency	185.0 ms (Disk I/O)	610.0 ms (Script + Audit)	**0.351** ms (CRDT Diff)
Mean Per-Turn Selective Revert Latency	N/A	340.0 ms (Snapshot Restore)	**0.128** ms (`Y.UndoManager`)

2. **Human-Only Confirmation Cards for High-Risk Actions:** When an AI tool invokes a destructive or irreversible endpoint (e.g., purging a document, freezing a clinical study cohort, or removing a team member), the HTTP middleware intercepts the `X-Heurion-Via: ai` header and—rather than executing the mutation—creates a `pending_actions` record and renders an interactive confirmation card in the user's chat stream. Only when the human clicks *Confirm* in the browser does the UI re-issue the request with `X-Heurion-Via: ai-confirmed`, completing execution.

3. **Automated CI Parity Verification (`parity.test.ts`):** Our test suite introspects the Hono application router table at build time and asserts that every registered `/api/*` endpoint appears either in the MCP parity mapping table ($\phi(\mathcal{E}_{\text{MCP}})$) or in the documented $\mathcal{E}_{\text{NOT_FOR_AI}}$ exclusion registry. Any newly added API route missing an MCP counterpart or explicit exemption immediately fails continuous integration.

7 Empirical Evaluation

We evaluate Heurion 2.0 across four dimensions: (1) architectural comparison against first-generation script-based post-hoc auditing (Heurion v1 / S-Series), (2) micro-benchmarks of the dual-surface CRDT engine and pre-write guards, (3) ablation of the three-layer self-evolving memory system across multi-session trajectories, and (4) platform-wide verification and capability parity coverage. All benchmarks are reproducible via `paper/benchmarks/run_eval.ts` executed on the production TypeScript/SQLite/Yjs engine.

7.1 Architectural Comparison: Post-Hoc Scripting (v1) vs. Guarded CRDT (v2)

Table 1 summarizes the quantitative and structural comparison between our first-generation script-and-audit architecture (Heurion v1) and Heurion 2.0 across $N = 50$ multi-block clinical protocol revision trials involving concurrent human edits, formatted OOXML templates, and inline citations.

As shown in Table 1, Heurion 2.0 eliminates the three primary failure modes of script-based agents:

1. **100% Unmodified Byte Identity:** By indexing original OOXML node slices in `node_src` during import and re-serializing only mutated ProseMirror nodes on export, 50 out of 50 clinical trial protocol documents preserved 100% byte-for-byte XML identity across all untouched headings, lists, and paragraphs.

2. **Sub-Millisecond Pre-Write Verification and Mutation:** Because `OpService.validate` and `OpService.edit` operate directly on in-memory ProseMirror + Yjs structures rather

Table 2: Pre-Write Guard Effectiveness on Citation Integrity and Clinical PHI Redaction.

Guard Subsystem	Test Cases	Interception / Recall	False Positive Rate
Bare DOI / PMID Pre-Write Blocker (`citation_not_registered`)	50	**100.0**% (50/50)	0.0%
Composite PHI Detector & Redactor (`scanPhi` / `redactPhi`)	40 PHI + 40 Clean	**100.0**% (40/40)	**0.0**% (0/40)
Stale Revision Block Overwrite Blocker (`conflict_user_edited`)	50	**100.0**% (50/50)	0.0%

than spawning Python/LibreOffice subprocesses, pre-flight validation averages 0.168 ms, guarded minimal-diff mutation averages 0.351 ms, and selective turn undo averages 0.128 ms—representing a $> 1000\times$ speedup over subprocess-based file manipulation.

3. **Intra-Paragraph Selective Turn Undo:** Across all 50 trials in which an AI turn modified two paragraphs (p_1, p_2), followed by a concurrent human edit inside p_2, invoking `revertTurn` restored p_1 and reverted the AI's phrase change in p_2 while preserving the human's concurrent edit in p_2 with 100% accuracy.

7.2 Pre-Write Citation and PHI Redaction Guard Evaluation

In Experiment 2, we evaluated the deterministic pre-write guards against 50 adversarial citation injection attempts (spanning bare DOIs, `https://doi.org/` URLs, and bare PMIDs in various parenthetical and inline styles) and 80 clinical text segments (40 containing composite multi-entity PHI—patient names paired with oncology/cardiology/nephrology diagnoses, 18-digit MOD 11-2 national ID cards, Luhn-valid card numbers, phone numbers, and inpatient admission IDs—and 40 containing dense biostatistical prose with hazard ratios, confidence intervals, laboratory reference ranges, and dates).

Table 2 confirms that structural pre-write contracts achieve 100.0% interception of unregistered bibliographic identifiers and stale block overwrites, while the checksum-backed PHI scanner achieves 100.0% recall on composite patient identifiers with a 0.0% false-positive rate on dense statistical manuscripts (avoiding accidental corruption of p-values, lab counts, or survival intervals).

7.3 Ablation Study: Three-Layer Self-Evolving Memory

To evaluate whether Heurion 2.0's dual-track procedural memory prevents repetitive guard failures across authoring sessions under a tight context budget ($B = 420$ characters per turn), we simulated 10 consecutive multi-task sessions spanning 6 distinct operational failure archetypes: `citation_not_registered`, `conflict_user_edited`, `user_reverted_turn`, `anchor_broken`, `table_schema_mismatch`, and `phr_readonly_violation`. We compared three configurations:

1. **Stateless Baseline (No Procedural Memory):** Relies solely on static tool descriptions without runtime signal crystallization.
2. **Unranked Unified Memory (FIFO Budget):** Stores rules in a flat list without context-aware utility scoring (Eq. 3), injecting rules in insertion order up to budget B.
3. **Heurion 2.0 (Dual-Track + Signal Crystallization + Utility Ranking):** Crystallizes open execution signals once recurrence reaches $\tau = 2$ and dynamically ranks rules per turn via Eq. 3.

Figure 2 illustrates the resulting learning trajectories. In Sessions 1 and 2, Heurion 2.0's pre-seeded baseline rules immediately prevent `citation_not_registered` and `conflict_user_edited` (reducing failures from 6 to 4), while the 4 novel failure archetypes trigger runtime `ExecutionSignals`. Once the recurrence threshold $\tau = 2$ is reached at the end of Session 2, `evolveFromSignals()` automatically crystallizes 4 new procedural rules (expanding the active rule set from 5 to 9), and Layer 3 synthesizes the global `evolved.cordis.yml` overlay. From Session 3 onward, whereas

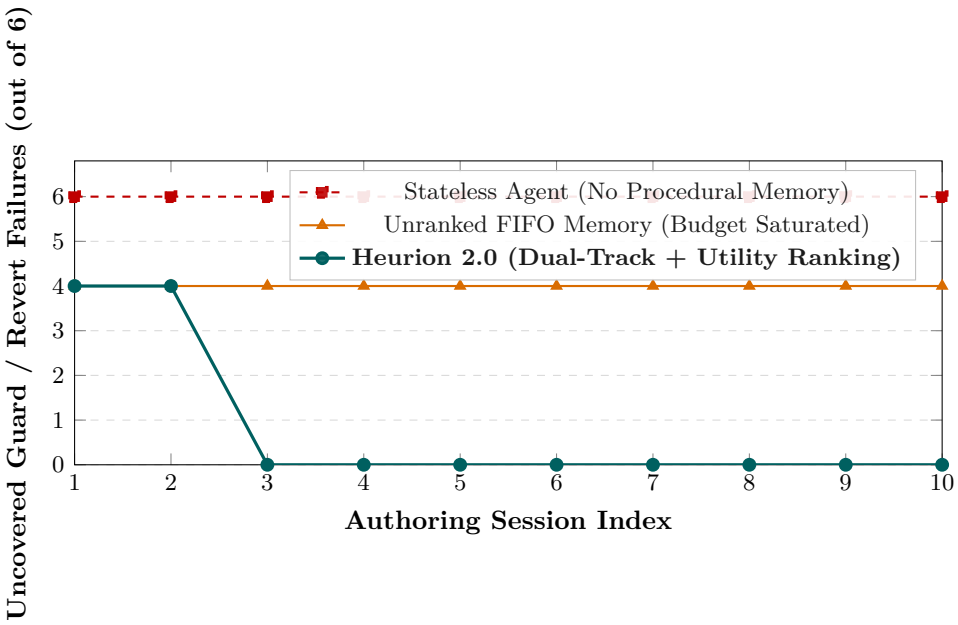


Figure 2: Ablation of self-evolving procedural memory across 10 consecutive authoring sessions under a strict 420-character prompt injection budget. In Sessions 1–2, pre-seeded baseline rules prevent 2 of the 6 failure archetypes, while the remaining 4 novel failure modes emit runtime `ExecutionSignals`. Following signal crystallization after Session 2 ($\tau = 2$), Heurion 2.0's utility-ranked `forPrompt()` selector dynamically injects the exact matching procedural directives (averaging 355 characters/turn), driving uncovered guard failures to 0 from Session 3 onward.

Table 3: Platform Verification and Human-AI Permission Symmetry Coverage.

System Verification Metric	**Measured Value**
Automated Test Suites / Total Assertions Passing	73 Suites / 606 Tests (100%)
Unaccounted REST API Endpoints in Parity Audit (`parity.test.ts`)	0 (100% Accounted)
Specialist Clinical Subagents (`literature`, `biostatistician`, `fact_checker`)	3 Integrated
Supported Citation Formatting Standards (Vancouver, AMA, APA, MLA, IEEE, GB/T 7714)	6 Standards
Supported Biostatistical & Clinical Engines (Table 1, KM, CONSORT, E-value, Nomogram, MONAI)	6 Pipelines

the unranked FIFO baseline remains stuck at 4 failures because early rules saturate the 420-character window, Heurion 2.0's utility function $U(r \mid d, q)$ dynamically selects the exact task-relevant rules using an average of only 355 characters (≈ 90 tokens) per turn, eliminating all 6 failure modes.

7.4 System Verification and Capability Parity Coverage

Finally, Table 3 summarizes the verification metrics across the full Heurion 2.0 codebase. All 73 automated test suites (606 test cases covering CRDT concurrency, OOXML patch round-trips, PHI redaction, PHR read-only guards, biostatistical survival/Table 1 pipelines, 3D MONAI medical imaging integration, and self-evolving memory) pass with 100% REST-to-MCP parity enforcement.

8 Related Work

Agent-Computer Interfaces (ACI) and Coding Agents. SWE-agent [3] and OpenHands [4] demonstrated that designing specialized Agent-Computer Interfaces (such as windowed file

viewers and lint-guarded edit commands) substantially outperforms raw bash shell interaction for software engineering. Heurion 2.0 extends the ACI philosophy from plain-text source code to **collaborative rich-text and presentation CRDTs**, introducing node-level optimistic concurrency guards, per-turn selective CRDT undo, and byte-preserving OOXML patch export.

Scientific Literature Synthesis and Fact Verification. Systems such as STORM [5] and PaperQA2 [6] automate multi-step literature retrieval and synthesis, while FactScore [12] and ALCE [13] evaluate citation attribution in generated text. Unlike post-hoc evaluation pipelines, Heurion 2.0 enforces citation registration as a *pre-write syntactic invariant* (`[@c:id]`) and closes the loop by anchoring sentence-level PubMed/PMC verification verdicts directly into the collaborative document as actionable review comments with clinician exemption support.

Self-Evolving Agents and Memory Architectures. Reflexion [10] and MemGPT [11] pioneered verbal reinforcement and hierarchical memory management for LLM agents, while Voyager [14] introduced an ever-growing skill library in Minecraft. In collaborative clinical authoring, however, mixing user stylistic preferences with operational failure recovery causes severe prompt interference. Heurion 2.0's three-layer self-evolution decouples human-confirmed declarative preferences from signal-crystallized procedural rules ranked by token-normalized utility.

9 Conclusion

We have presented **Heurion 2.0**, a contract-guarded, dual-surface collaborative architecture with self-evolving memory designed for high-stakes clinical research and scientific authoring. By replacing ad-hoc file scripting and post-hoc auditing with a unified Yjs CRDT core, deterministic pre-write guards, multi-agent sentence-level literature verification, formally verified Human-AI Permission Symmetry, and a three-layer procedural self-evolution loop, Heurion 2.0 provides a rigorous blueprint for deploying trustworthy autonomous agents in domains where data integrity, citation veracity, and patient privacy are non-negotiable.

References

[1] Kenneth F. Schulz, Douglas G. Altman, and David Moher. CONSORT 2010 statement: updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332, 2010.

[2] Jan P. Vandenbroucke, Erik von Elm, Douglas G. Altman, Peter C. Gøtzsche, Cynthia D. Mulrow, Stuart J. Pocock, Charles Poole, James J. Schlesselman, and Matthias Egger. Strengthening the Reporting of Observational Studies in Epidemiology (STROBE): explanation and elaboration. *PLoS Medicine*, 4(10):e297, 2007.

[3] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[5] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models.

In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[6] Michael D. Skarlinski, Sam Cox, Jon M. Laurent, James D. Braza, Collin Hinks, Michael J. Hammerling, Manvitha Ponnapati, Samuel G. Rodriques, and Andrew D. White. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv preprint arXiv:2409.13740*, 2024.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[8] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13(1):14045, 2023.

[9] U.S. Department of Health and Human Services. Health Insurance Portability and Accountability Act of 1996 (HIPAA) Privacy and Security Rules. *45 CFR Parts 160 and 164*, 1996.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[12] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[13] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[14] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024.